ARCHITECTURE REFERENCE · POWEROPS_V1

PowerOps

A retrieval-augmented DevOps management assistant that answers natural-language questions about real Jira-style issue data across three teams — grounded strictly in retrieved evidence, and honest when it isn't sure.

Falcon Squad, Nova Team, and Summit Crew each track their DevOps work as Jira issues — CI/CD failures, certificate renewals, database access requests, Dynatrace alerts, deployment hotfixes. Answering a question like *"What high-priority problems belong to Nova Team?"* today means opening a Jira filter and reading through tickets by hand. PowerOps answers it directly, in a sentence or a table, and cites the exact `Issue Key`s it used as evidence.

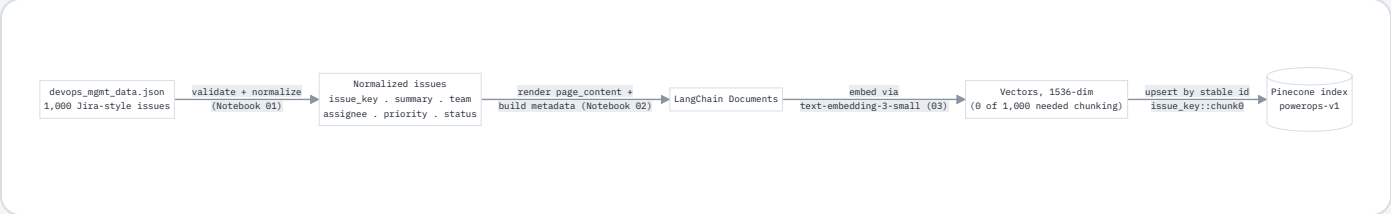
The system is built as a hybrid retriever, not a pure semantic-search demo: a deterministic parser reads structured intent out of the question — team, assignee, priority, status, issue key — and turns it into an exact Pinecone metadata filter, while whatever's left of the question drives the semantic side of the search. When the evidence genuinely isn't there, PowerOps says so and opens a management escalation instead of guessing.

1,000	3	62	6	6	0
real issues indexed	teams	distinct assignees	notebooks, sequential	evidence checks	hallucinations observed

§ Architecture & flow

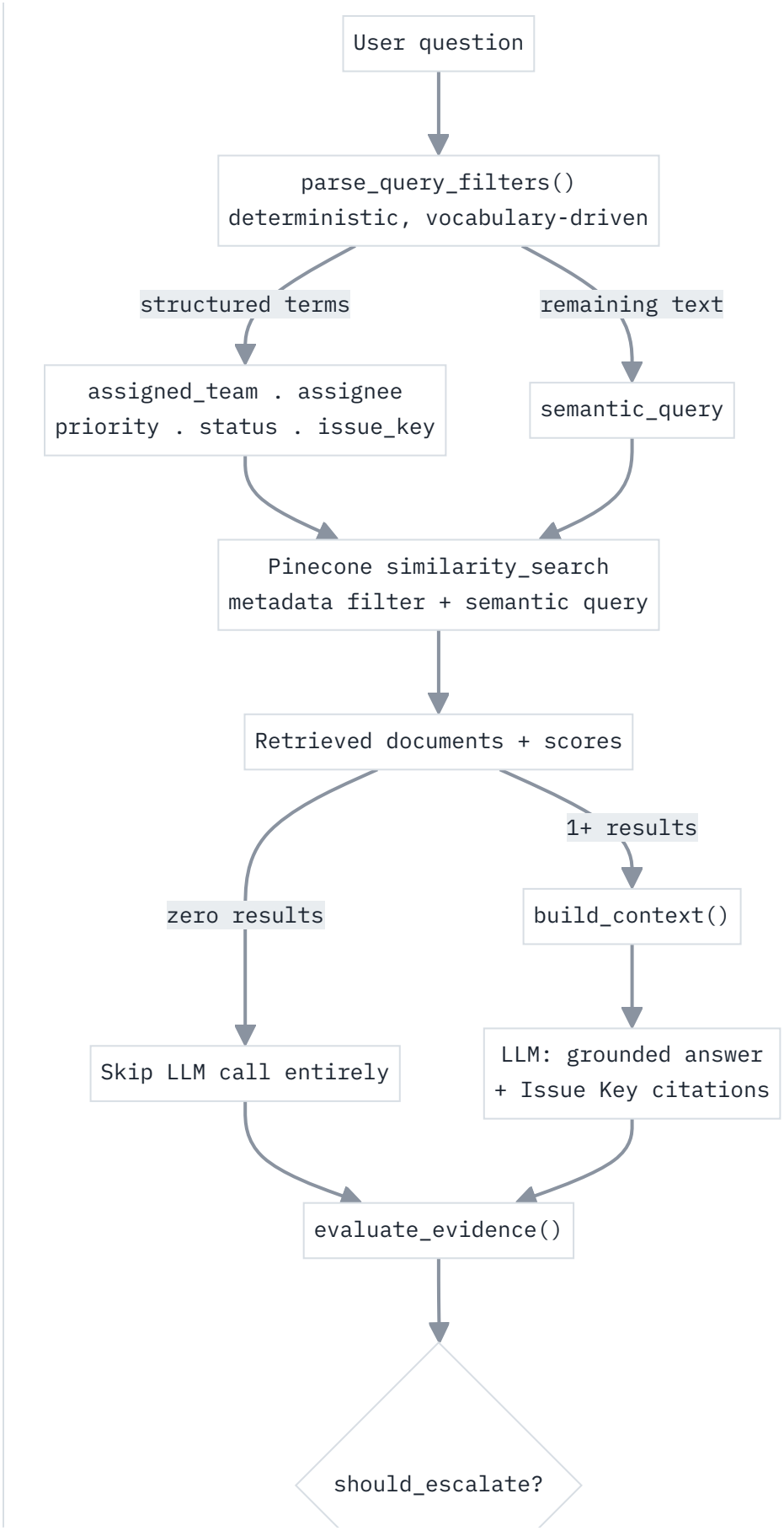
Two distinct pipelines make up PowerOps: an **ingestion pipeline** that runs once (and idempotently, on re-run) to get the knowledge base into Pinecone, and a **query-time pipeline** that runs on every question — combining metadata filtering, semantic search, grounded generation, and evidence-based escalation.

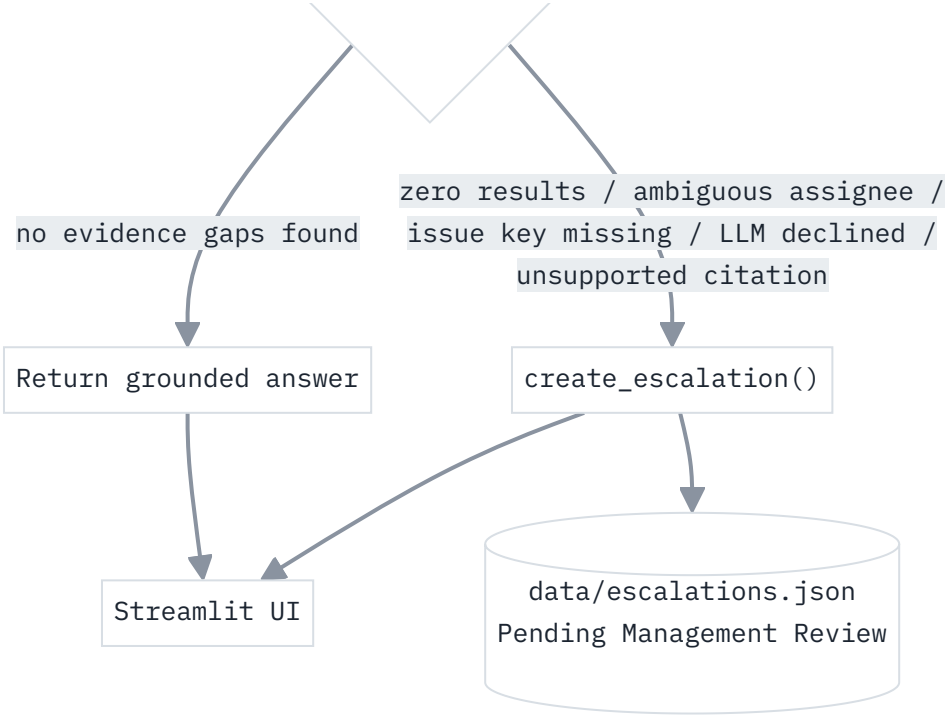
Ingestion — building the knowledge base



Runs once per data refresh. Stable vector IDs make re-running ingestion an update, not a duplicate.

Query time — hybrid retrieval, grounding, escalation


User question
parse_query_filters()
deterministic, vocabulary-driven
structured terms
remaining text
assigned_team . assignee
priority . status . issue_key
semantic_query
Pinecone similarity_search
metadata filter + semantic query
Retrieved documents + scores
1+ results
zero results
build_context()
Skip LLM call entirely
LLM: grounded answer
+ Issue Key citations
evaluate_evidence()
should_escalate?



Every answer — not just the zero-result case — passes through `evaluate_evidence()` before being returned.

WHY HYBRID, NOT PURE SEMANTIC SEARCH

Notebook 03's test queries showed cosine similarity in the 0.35–0.55 range even for genuinely relevant matches — this dataset's Summaries are short and terse, which compresses the similarity signal. There is nothing semantically distinctive about the string "Nova Team" that would make an embedding model reliably rank Nova Team's issues above the other two teams' for an otherwise-generic question. Team, assignee, priority, and status are categorical facts, not fuzzy concepts — they need exact filtering, not similarity ranking.

01 Data loading & exploration

01 Load, validate, normalize, profile

notebooks/01_data_loading_and_exploration.ipynb

Loads the raw Jira export and maps its actual field names onto a clean internal schema — the source data's real shape differs from a generic ticket template in ways that matter downstream:

RAW FIELD	INTERNAL FIELD	NOTE
`Issue key`	`issue_key`	Primary identifier, e.g. `INO-21920`
`Summary`	`summary`	Only free-text field — no description, comments, or resolution exist
`Custom field (Assigned Team)`	`assigned_team`	Falcon Squad, Nova Team, Summit Crew
`Priority`	`priority`	Low / Medium / High / Urgent — no "Critical" tier
`Status`	`status`	To Do / In Progress / Done / Rejected / On Hold / Soft Delete
`Updated, Due date`	`updated_dt, due_date_dt`	Parsed to datetimes; blanks become None, never guessed

RESULT

1,000 records, **0 duplicate issue keys, 0 invalid records**. Team split: Nova Team 438 · Falcon Squad 365 · Summit Crew 197. Priority split: Medium 879 · High 91 · Urgent 27 · Low 3. Outputs `data/normalized_issues.json` and `data/vocabulary.json` — the latter becomes the parser's real vocabulary in Notebook 04, instead of hardcoded guesses.

02 Document preparation

02 Build Documents, decide chunking

notebooks/02_document_preparation.ipynb

Each issue becomes one `Document`: readable text for embedding, structured metadata for filtering.

PAGE_CONTENT

RENDERED TEXT BLOCK

```
Issue Key: INO-21920
Summary: SDX case are creating but remaining in Delayed Processing Pending status -
FT13
Assigned Team: Falcon Squad
Assignee: Sullivan, Deepa (Contractor)
Reporter: Sullivan, Deepa (Contractor)
Priority: Medium
Status: In Progress
Story Points: 0.25
Last Updated: 5/18/26 12:01
Due Date: 5/15/26 0:00
```

METADATA

`issue_key`, `assigned_team`, `assignee`, `priority`, `status`, `story_points`, `updated_epoch`, `due_date_epoch` (numeric, for future range-filter questions), plus `has_due_date` / `has_story_points` presence flags so "no due date" is distinguishable from a coerced default.

RESULT

Document lengths ranged 216–393 characters (mean 279) — **0 of 1,000** exceeded the 1,500-char chunking threshold. Chunking via `RecursiveCharacterTextSplitter` is implemented and metadata-preserving regardless, so a future longer-text issue is handled safely. Stable vector IDs (`issue_key::chunk0`) make Pinecone upserts idempotent.

03 Pinecone ingestion

03 Embed, index, verify

notebooks/03_pinecone_ingestion.ipynb

Configuration is entirely environment-driven (`.env`) — embedding provider, model, and dimensions; Pinecone index name, cloud, and region; chat model and temperature — so a provider swap never touches notebook code.

- Embeds with the configured model, **verifies the actual output dimension** against `.env` before touching Pinecone, and fails fast with a clear message on mismatch.
- Creates the serverless index only if it doesn't already exist.
- Upserts in batches of 100 using precomputed vector IDs — re-running this notebook updates vectors, it doesn't duplicate them.
- Verifies the resulting vector count matches the document count.

RESULT

1,000 / 1,000 vectors confirmed in `powerops-v1` (1536-dim, cosine, aws/us-east-1). Test queries such as *"certificate failures"* correctly surfaced `IN0-20582` ("Cert IDPPNCID2... Expires: 03/27/2027") — but at moderate similarity (~0.42), confirming the compressed-score behavior that motivates the hybrid design.

04 Metadata-aware retrieval

04 parse_query_filters() + retrieve_powerops_documents()

`notebooks/04_metadata_retrieval.ipynb`

A fully deterministic parser — no LLM call, no synonym guessing — matched against the *real* vocabulary extracted in Notebook 01.

- **Issue key:** strict `INO-\d+` regex.
- **Team / priority / status:** whole-phrase, case-insensitive match against the actual dataset values only.
- **Assignee — the hard part:** stored as `"Last, First (Contractor)"`; a two-tier matcher tries a full-name match first, then falls back to a single-token match — and if a token matches *multiple* real people, that's surfaced as genuine ambiguity rather than guessing one of them.

QUESTION	DETECTED
"high priority issues assigned to Falcon Squad"	• `assigned_team=Falcon Squad, priority=High`
"issues does Gibson, Anjali have"	• `assignee=Gibson, Anjali` (unambiguous)
"issues does Anjali have"	• `ambiguous — 3 real people match`
"show me all critical issues"	• `no priority match` — "Critical" isn't a real value
"issue OPS-1024"	• `no issue_key match` — wrong prefix, doesn't exist

DELIBERATE DESIGN CHOICE

Matching is **literal-only**. "Critical" and "open"/"blocked" produce no filter because this dataset genuinely has no such values — the gap surfaces honestly as reduced precision instead of being papered over with an invented synonym mapping. An optional LLM-based structured-output parser is demonstrated for comparison in this notebook, constrained to the same real vocabulary, but the deterministic parser is what production retrieval actually uses.

05 RAG question answering

05 ask_powerops() v1

`notebooks/05_powerops_rag.ipynb`

Wraps Notebook 04's retrieval with context construction and a grounding-focused system prompt (full text in § Prompts in use). If retrieval returns zero documents, the LLM is never called — there's nothing to ground an answer in, so the "insufficient evidence" response is returned directly.

RESULT — AND THE GAP IT EXPOSED

Across 10 test questions, **zero hallucinations** — the model correctly declined on 4 deliberately unanswerable questions and 3 more where retrieval returned real-but-irrelevant documents. But `needs_escalation` stayed `False` in every one of those 7 cases, because the placeholder rule only checked for zero retrieved documents — never for "the LLM's own answer said it found nothing." That gap is exactly what Notebook 06 closes.

06 Confidence & human escalation

06 evaluate_evidence() · should_escalate() · create_escalation()

notebooks/06_human_escalation.ipynb

Six deterministic, checkable signals — none of them a self-reported confidence score:

1. **Zero results** retrieved at all.
2. **Requested issue key not found** among what came back.
3. **Requested filter combination not reflected** in any retrieved record (a defensive re-check on top of the Pinecone filter itself).
4. **Ambiguous assignee** — a name matches more than one real person.
5. **LLM indicated insufficient context** — checked by matching the answer text against known decline phrases, not by asking the model a second "are you sure?" question.
6. **Unsupported citations** — the answer cites an Issue Key that isn't actually in the retrieved context.

DELIBERATELY NOT USED: RAW SIMILARITY THRESHOLD

This dataset's short Summaries compress cosine similarity into a narrow band (~0.35–0.55) even for good matches, so a fixed cutoff would either miss real problems or flag good matches as weak. `max_similarity` is reported for visibility only, never used as a trigger on its own.

RESULT

Re-running Notebook 05's 5 broken cases: all now correctly escalate — 4 on "LLM indicated insufficient context," 1 (the Anjali question) on "ambiguous assignee." Notably, on that last case the LLM's own free-text answer still confidently framed the results as *"issues related to Anjali Porter"* specifically — despite the table it generated containing a *different* Anjali and even a non-Anjali assignee. The deterministic ambiguity check caught this correctly even though the generated prose was subtly misleading — exactly why evidence checks don't trust the model's own framing.

§ Streamlit application

`app.py` wires the full pipeline into an interactive UI: a question box, an **Ask PowerOps** button, a grounded answer (rendered as markdown, so LLM-generated tables display natively), and — when `should_escalate()` fires — a red **Management Escalation Required** panel with a real Escalation ID and status. Three expandable sections (Retrieved Sources, Applied Filters, Retrieval Details) show the mechanics behind any answer. Pinecone/embedding/LLM clients are built once via `st.cache_resource`, not reconnected on every rerun.

Verified end-to-end with a headless browser: a Nova Team high-priority question produced a clean grounded table with a green "no escalation needed" banner; a question about a person not in the dataset produced the red escalation panel with a real ESC-##### ID written to `data/escalations.json` — zero console errors in either case.

§ Prompts in use

GROUNDING SYSTEM PROMPT — ASK_POWEROPS()

USED ON EVERY ANSWER GENERATION CALL

```
You are PowerOps, a DevOps management assistant.

Answer the user's question using ONLY the supplied PowerOps context below.

Do not invent issues, statuses, assignees, priorities, teams, dates, or
resolutions that are not explicitly present in the context.

When referencing an issue, include its Issue Key (format: INO-#####).

If the available context does not contain enough evidence to answer the
question, explicitly state that sufficient information was not found in
the PowerOps knowledge base. Do not guess or fill gaps.

For questions requesting multiple issues, provide a concise table when
appropriate (columns: Issue Key, Summary, Team, Assignee, Priority, Status).
```

OPTIONAL STRUCTURED-OUTPUT PARSER — COMPARISON ONLY (NOTEBOOK 04)

NOT USED IN THE PRODUCTION PATH — DETERMINISTIC PARSER WINS ON PREDICTABILITY

```
You extract structured filters from a DevOps question for PowerOps.
Only use these exact known values (case-sensitive) - never invent new ones:
Teams: ['Falcon Squad', 'Nova Team', 'Summit Crew']
Priorities: ['High', 'Low', 'Medium', 'Urgent']
Statuses: ['Done', 'In Progress', 'On Hold', 'Rejected', 'Soft Delete', 'To Do']
If a concept in the question (e.g. 'critical', 'open', 'blocked') does not
exactly match one of the known values above, leave that field null rather
than guessing the closest one.
```

§ Design decisions

- **Deterministic parsing over LLM parsing.** Zero per-query cost or latency, and structurally cannot invent a team/priority/status that doesn't exist in the data.
- **Literal vocabulary matching, no synonyms.** A question asking for "critical" issues gets no priority filter, honestly, rather than a guessed mapping to "Urgent."
- **Ambiguity is surfaced, never silently resolved.** Three different "Anjali"s exist in this data; the parser says so instead of picking one.
- **Zero-document short-circuit.** No context, no LLM call — cheaper and more certain than hoping the model self-reports uncertainty correctly.
- **Evidence checks are deterministic, not confidence-based.** Every escalation trigger is a checkable fact about the retrieval or the answer text — never "how sure did the model sound."
- **Idempotent ingestion.** Stable, content-derived vector IDs mean re-running Notebook 03 updates the index rather than duplicating it.

§ Known limitations

- Summary-only text compresses semantic similarity into a narrow ~0.35–0.55 band even for strong matches — a real ceiling on ranking quality for this specific dataset.
- No description, comments, or resolution fields exist in the source data; `page_content` is Summary + metadata only.
- Assignee names can be genuinely ambiguous across real people sharing a first name — surfaced to the user, not resolved automatically.
- An LLM's free-text framing can be subtly misleading even when the underlying escalation decision is correct (see the Anjali Porter finding, §06) — a reminder that generated prose is never itself an evidence signal.
- The escalation log has no deduplication — identical repeated questions create repeated records.
- Single-turn only: no conversational memory or "which of those…" follow-up support yet.
- Retrieval/RAG/escalation logic is currently duplicated across Notebooks 04–06 and `app.py`, pending a planned `src/` extraction.

§ Roadmap

- → **Extract src/ modules**
  Pull the logic duplicated across Notebooks 04–06 and app.py into config.py, query_parser.py, retriever.py, rag_chain.py, escalation.py — notebooks and the app import shared code instead of copy-pasting it.

- → **Operations dashboard & escalation review**
  A management view: issue counts by team/status/priority/assignee, and a queue for reviewing pending escalations with status transitions (Pending → In Review → Answered → Closed).

- → **Bounded conversational follow-ups**
  Support "which of those are assigned to Sarah?" via session state — while every answer still re-grounds against Pinecone rather than treating a prior LLM turn as fact.

- → **Escalation integrations**
  Push created escalations into Jira, ServiceNow, Teams, or Slack instead of a local JSON file only.

- → **Similarity calibration or reranking**
  A reranking pass or dataset-calibrated thresholds to make "weak match" a usable signal despite this data's compressed similarity range.

- → **Regression evaluation harness**
  A fixed set of question → expected grounding/escalation behavior, run automatically so future changes can't silently reintroduce the Notebook 05 escalation gap.

- → **Auth, multi-user attribution, escalation dedup**
  `create_escalation()` already accepts a `user` field, currently always null; wiring real auth also unlocks deduplicating repeated identical escalations.

PowerOps_v1 — built across Notebooks 01–06 on the real Falcon Squad / Nova Team / Summit Crew dataset, plus a verified Streamlit interface.